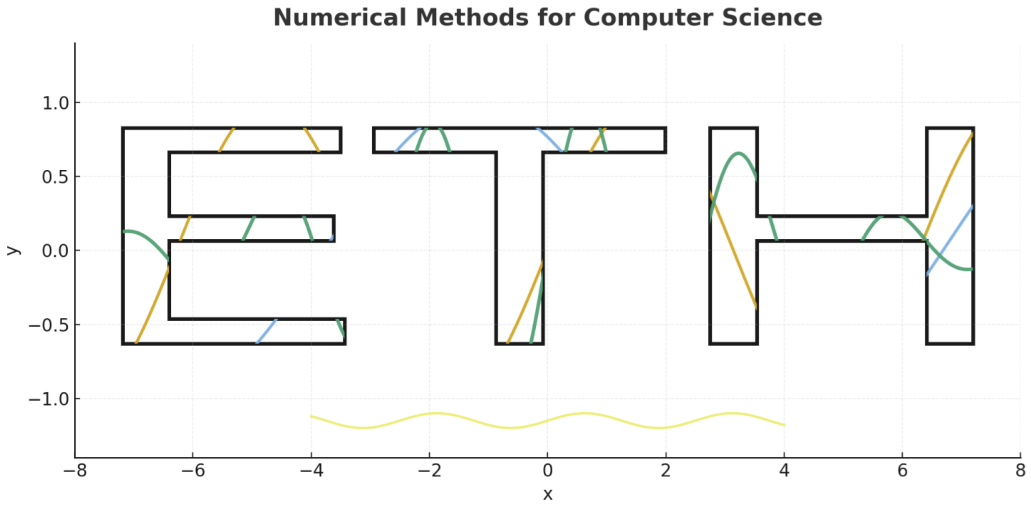

Advanced Topics in Numerical Quadrature — Week #7

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This week continues our study of *Numerical Quadrature*, focusing on the remaining topics, from convergence analysis and composite formulas to the Romberg scheme and spectral approaches. We move beyond the classical midpoint, trapezoidal, and Simpson rules toward adaptive, higher-order, and FFT-based methods.

Contents

1 Review and Transition from Week 6

We defined Numerical quadrature as the art of approximating an integral by a finite weighted sum. It is one of the cornerstones of computational mathematics. Last week, we focused on the fundamental ideas that allow us to estimate the area under a curve using simple geometric reasoning: rectangles, trapezoids, and parabolas. This week we move beyond the elementary rules and explore more advanced integration techniques that improve precision and computational efficiency.

1.1 From Simple to Composite Integration Formulas

In the previous lecture, we derived three classical formulas:

$$Q_M(f;a,b) = (b-a)f\left(\frac{a+b}{2}\right), \quad Q_T(f;a,b) = \frac{b-a}{2}\,[f(a)+f(b)], \quad Q_S(f;a,b) = \frac{b-a}{6}\,[f(a)+4f(\tfrac{a+b}{2})+f(b)].$$

These formulas approximate the integral

$$I = \int_a^b f(x)\,dx$$

using only a few function evaluations. However, as we saw in Week 6, a single application over a large interval produces only limited accuracy, especially when $f(x)$ varies significantly.

Note

The key idea is to split the interval $[a,b]$ into smaller subintervals of size $h = \frac{b-a}{N}$ and apply the basic rule to each piece. This yields a *composite quadrature formula* that achieves a much better approximation with minimal extra cost.

Formally, for the Trapezoidal Rule:

$$Q_T^{(N)}(f;a,b) = \frac{h}{2}\left[f(x_0) + 2\sum_{k=1}^{N-1} f(x_k) + f(x_N)\right],$$

where $x_k = a + kh$. By increasing N, we reduce the step size h, and the global error decreases roughly as $O(h^2)$.

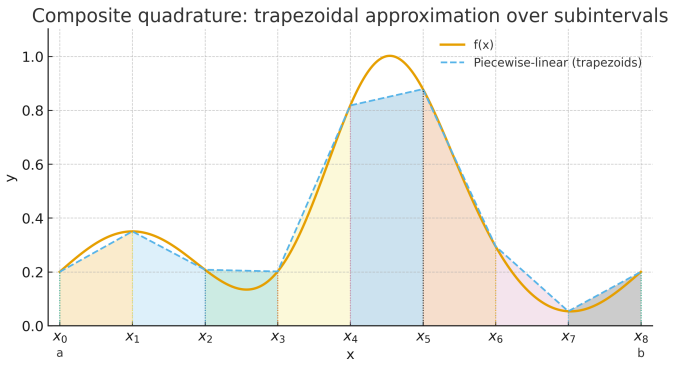

Figure: Composite quadrature subdivides the interval and reuses simple rules on each piece.

This principle ("divide and conquer") generalizes really good: local accuracy can be combined systematically to achieve global precision. It is the foundation for all modern integration schemes.

1.2 Revisiting Accuracy and Convergence Order

Every quadrature rule has an associated *order of accuracy* that describes how the error behaves when the step size h becomes small.

If a quadrature rule $Q_h(f)$ satisfies

$$\left| \int_a^b f(x)\,dx - Q_h(f) \right| = Ch^p,$$

we say that the method has order p. For example: - the Midpoint and Trapezoidal Rules are second–order ($p = 2$), - Simpson's Rule is fourth–order ($p = 4$).

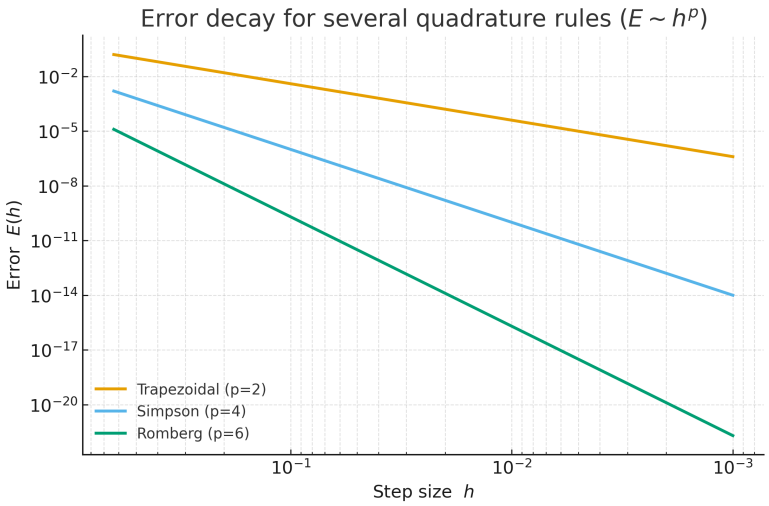

Figure: Error decay for several quadrature rules ($E \sim h^p$).

This quantitative notion of accuracy is crucial when balancing **cost vs. precision**. Each additional function evaluation requires computational effort, so the efficiency of a method depends not only on its order p but also on how much work it takes per subinterval.

> **Note**
>
> In practice, doubling the number of subintervals should ideally reduce the error by about 2^p. For Simpson's Rule ($p = 4$), halving h typically decreases the error by a factor of 16.

Higher–order rules are thus not just mathematically elegant, they can dramatically reduce computation time for a desired accuracy.

1.3 Why We Need Higher–Order and Adaptive Methods

Despite their elegance, the classical rules share a limitation: they rely on a *fixed* subdivision of the interval. This uniform spacing works well for smooth functions, but fails when $f(x)$ changes rapidly, exhibits singularities, or contains regions of very different behavior.

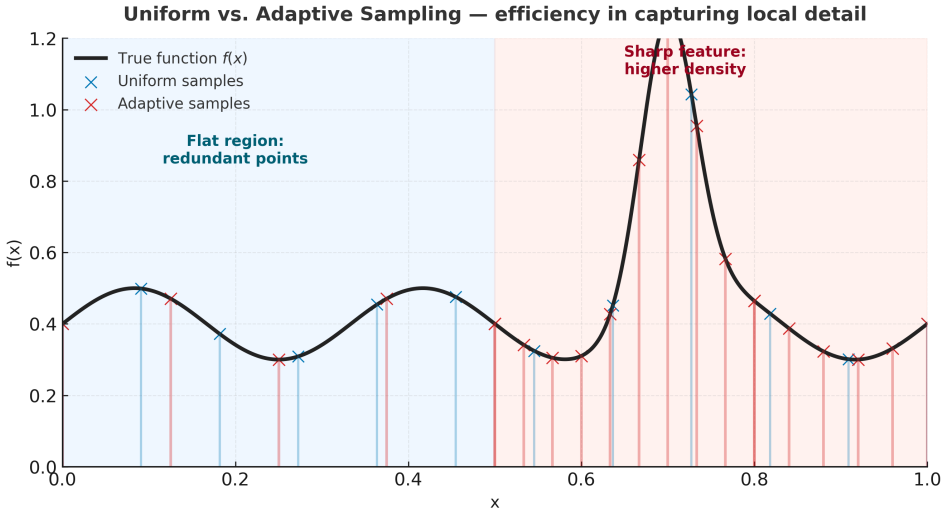

Figure: Uniform vs. adaptive sampling.Uniform grids waste effort in flat regions and miss sharp features.

To overcome this, modern numerical integration uses:
- **higher–order methods** (such as Romberg or Clenshaw–Curtis) that systematically increase precision without more function evaluations, and
- **adaptive methods** that refine the mesh dynamically, placing more nodes where the function varies most.

Both approaches share a common goal: **maximize accuracy per evaluation**. They form the bridge from classical numerical quadrature to the advanced, high–performance algorithms used in scientific computing today.

> **Note**
>
> This week we build directly on the concepts from Week 6, extending the theory toward convergence analysis and advanced methods such as Romberg integration and spectral quadrature. Keep in mind that every new method is, at its core, an optimized version of the same simple principle:
>
> $$\int_a^b f(x)\,dx \approx \sum_i \omega_i f(c_i).$$
>
> The art lies in choosing the right points c_i and weights ω_i.

2 Error Analysis and Convergence

Understanding how and why numerical quadrature approximations become accurate is crucial for both theory and practice. This section introduces the mathematical framework that quantifies error, explains how it behaves under refinement, and distinguishes between algebraic and exponential convergence.

2.1 Local vs. Global Error

When applying a quadrature rule to a function $f(x)$ on $[a, b]$, the total error arises from the accumulation of small local errors on each subinterval.

> Note
>
> **Local error**: the deviation produced by a single subinterval approximation.
> **Global error**: the total accumulated error after summing all local contributions.

Suppose the interval is divided into N equal parts, each of size $h = (b - a)/N$. If the local error on one piece behaves like $O(h^{p+1})$, then the global error, after summing over N subintervals, scales as:

$$E_{\text{global}} = N \cdot O(h^{p+1}) = O(h^p).$$

This relation explains why:

- **Trapezoidal Rule:** local error $O(h^3) \to$ global error $O(h^2)$;
- **Simpson's Rule:** local error $O(h^5) \to$ global error $O(h^4)$.

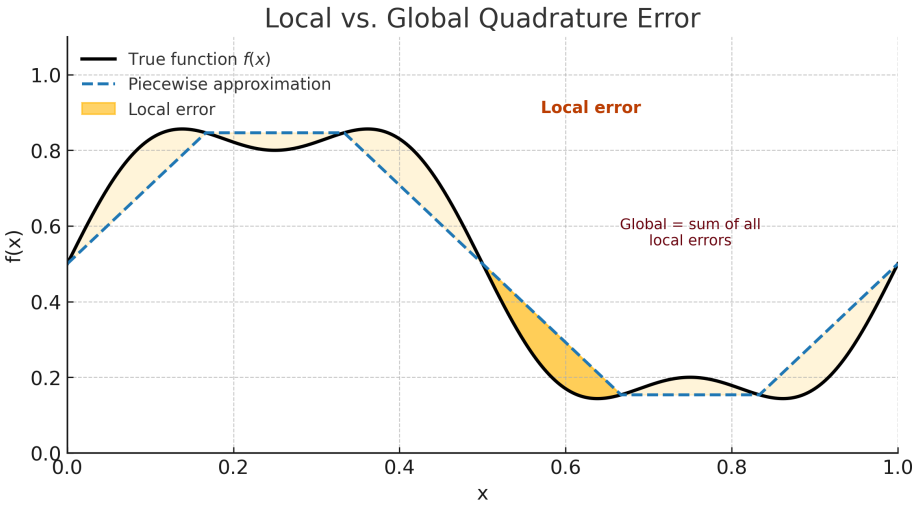

Figure: Local error on one subinterval accumulates into the global quadrature error.

2.2 Algebraic vs. Exponential Convergence

Most classical integration formulas exhibit **algebraic convergence**, meaning the error decreases polynomially with the step size h:

$$E(h) \sim C\,h^p, \quad p = \text{order of accuracy.}$$

This is the case for all rules derived from polynomial interpolation (as we have already seen: midpoint, trapezoidal, and Simpson's).

However, some special situations lead to much faster decay of the error:

$$E_n \sim C\,q^n, \quad 0 < q < 1,$$

which is called **exponential** (or *spectral*) convergence.

Here, the index n does not refer to the step size, but to the *number of sample points* (or degrees of freedom) used in the quadrature rule. Each additional point typically reduces the error by a constant factor $q < 1$, so the accuracy improves geometrically rather than algebraically.

Note

Exponential convergence typically appears under very smooth conditions:

- when the function $f(x)$ is **analytic**, meaning it can be represented by a (convergent) power series around every point in the interval. In simpler terms, such a function has derivatives of all orders and its graph is perfectly smooth, with no corners or discontinuities;
- or when both the function and the integration domain are **periodic**, so that the trapezoidal rule can exploit the symmetry and smooth repetition of the pattern.

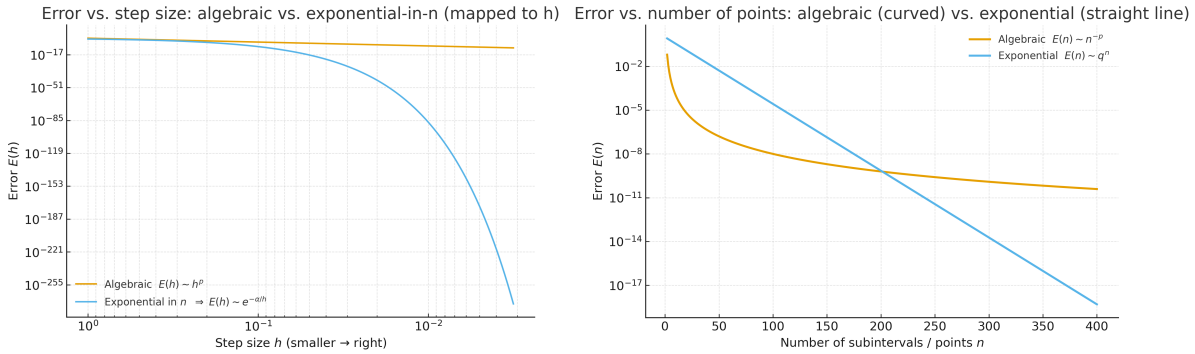

Figure: Algebraic vs. exponential convergence — comparison on $E(h)$ (left) and $E(n)$ (right) scales.

2.3 Effect of Function Smoothness

The smoothness of the integrand $f(x)$ largely determines the achievable accuracy. For smooth functions (with continuous derivatives up to order $p+1$), polynomial-based quadrature rules of order p perform optimally. But if $f(x)$ has discontinuities or sharp corners, convergence deteriorates dramatically.

$$\text{If } f \in C^{p+1}[a,b], \quad E(h) = O(h^p).$$

$$\text{If } f' \text{ or } f'' \text{ is discontinuous}, \quad E(h) \text{ decays much slower (often } O(h) \text{ or worse).}$$

Note

In practical terms:

- *Smooth regions* can be integrated accurately with large step sizes.
- *Non-smooth regions* require adaptive refinement or specialized quadrature.

This observation motivates the use of adaptive methods, where subintervals are refined only where the local error exceeds a threshold.

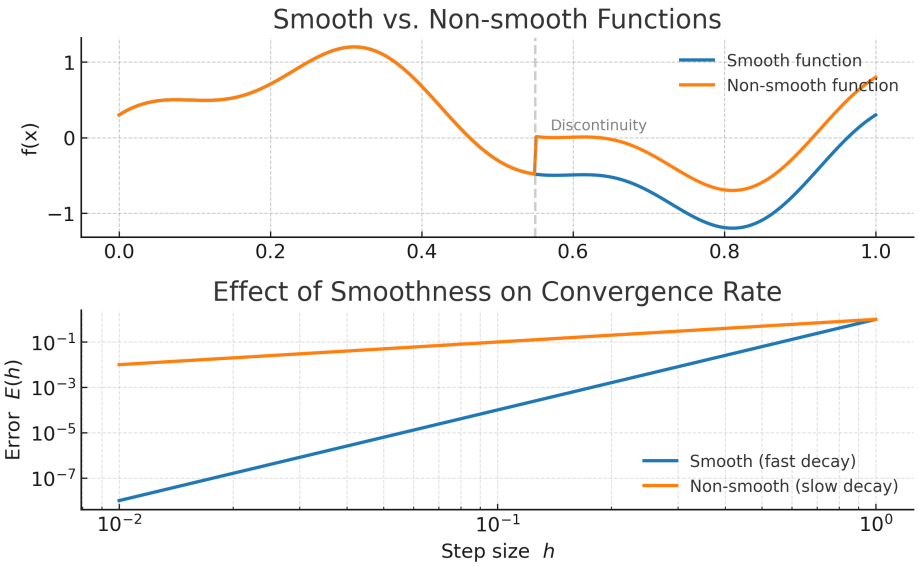

Figure: Smooth functions (blue) allow fast error decay; non-smooth functions (orange) converge slowly even with small h.

2.4 Visualizing Error Decay

We can now summarize the fundamental behaviors of error decay in quadrature methods. Let h denote the step size (or equivalently $n = 1/h$ the number of subintervals). Then we distinguish two typical regimes:

$$\textbf{Algebraic:} \quad E(h) \sim C\,h^p, \qquad \textbf{Exponential:} \quad E_n \sim C\,q^n,\ 0 < q < 1.$$

On a log–log scale, algebraic convergence appears as straight lines with slope p; exponential convergence appears as a near-vertical drop once n exceeds a small threshold.

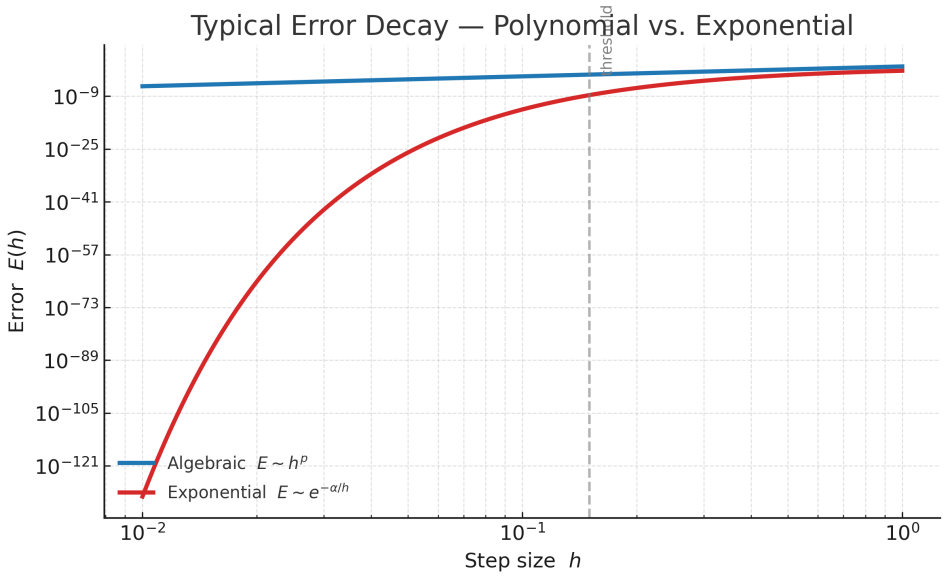

Figure: Typical error decay curves: polynomial (algebraic) vs. exponential.

Note

The goal of advanced quadrature methods (e.g., Romberg or Clenshaw–Curtis) is to accelerate algebraic convergence toward exponential behavior whenever the function regularity permits.

3 From Composite Rules to Smarter Accuracy Control

So far, we have seen that when the step size h is halved, the integration error decreases roughly by a predictable factor, according to the rule's order p:

$$E(h/2) \approx \frac{E(h)}{2^p}.$$

This simple relation opens the door to *smarter ways* of improving accuracy, without blindly refining the grid everywhere.

3.1 Key Idea: Predictable Error Ratios

When we apply a numerical rule with two different step sizes, for example h and $h/2$, we are essentially asking the same question twice: *"How close am I to the true value?"*

Both results, Q_h and $Q_{h/2}$, make the same kind of mistake: they just differ in *how large* that mistake is. Because we already know that the error shrinks roughly like h^p, we can compare the two and learn something about the remaining error.

- The two approximations Q_h and $Q_{h/2}$ are very close to each other, but not identical, so the difference between them reveals how fast the error is decreasing.
- By observing this pattern, we can **estimate** the remaining error or even **predict** what the result would be if h were infinitely small.
- In other words, we use the predictable way in which the error behaves to "see ahead" of our current grid resolution.

Note

This technique is called **extrapolation**. It works by detecting a consistent trend in the approximations and extending that trend toward the zero–error limit, without actually using a smaller h. It is a simple but powerful idea: instead of refining everywhere, we *learn from the error itself.*

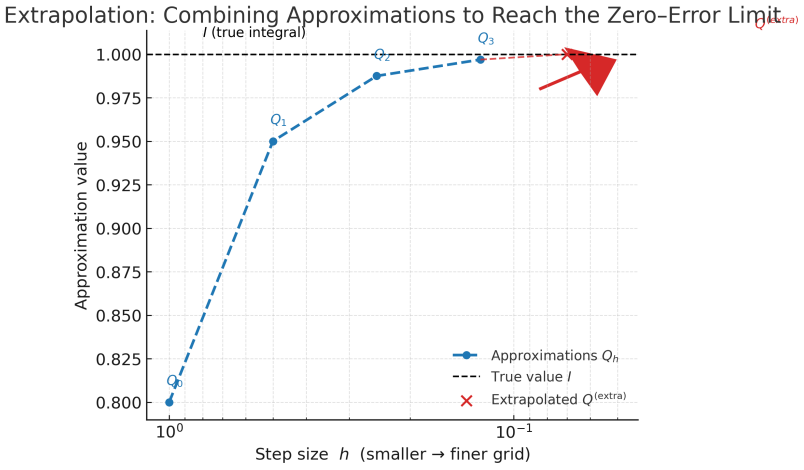

Figure: Successive approximations Q_h approach the true value I as h decreases; extrapolation (red)

extends this trend to estimate the zero–error limit.

3.2 Why This Matters - Toy example

Composite rules alone improve accuracy steadily, but slowly. Extrapolation methods, like the upcoming **Romberg integration**, use this predictable error behavior to accelerate convergence dramatically. They reuse the same function evaluations and produce high–order results with minimal extra work.

> Note
>
> Intuitively, extrapolation tells us: *"If I know how the error shrinks when I refine the grid, I can jump ahead and estimate what the result would be if the step size were infinitely small."*

A Simple Example: Seeing Extrapolation in Action Imagine that we are trying to approximate an integral I, but we do not know its exact value. We apply the same numerical rule twice: once with a coarse step size h, and once with a finer step $h/2$. This gives two results:

$$Q_h = 0.720, \qquad Q_{h/2} = 0.750.$$

The second result is closer to the truth, it used a smaller step, so its error is smaller. If the method we are using has order $p = 2$ (as with the trapezoidal rule), then halving the step should make the error roughly four times smaller, because $2^p = 4$.

That means the two numbers, Q_h and $Q_{h/2}$, contain almost the same type of error, just in different amounts. By comparing them, we can figure out what that error looks like and even *cancel* most of it.

To get closer to I, we can try to remove most of that error by combining the two results. Since $Q_{h/2}$ still has one–quarter of the error of Q_h, we can roughly "correct" it by adding one third of the difference between them:

Subtracting them gives:

$$Q_{h/2} - Q_h = \left(I - \frac{E}{4}\right) - (I - E) = \frac{3E}{4}.$$

So, the difference between the two results mainly represents the dominant error term, scaled by $\frac{3}{4}$ (now, observed that $\frac{1}{3} * \frac{3E}{4} = \frac{E}{4}$).

$$I \approx Q_{h/2} + \tfrac{1}{3}(Q_{h/2} - Q_h) = 0.750 + \tfrac{1}{3}(0.750 - 0.720) = \mathbf{0.760}.$$

$$I \approx Q_{h/2} + \frac{Q_{h/2} - Q_h}{2^p - 1} = 0.750 + \frac{0.750 - 0.720}{3} = \mathbf{0.760}.$$

Step size	Approximation	Comment
h	0.720	coarse grid (larger error)
$h/2$	0.750	finer grid (smaller error)
—	**0.760**	extrapolated (even higher accuracy)

Note

We did not compute any new function values or shrink the step size again, we simply *used the pattern of errors* from our two previous results. That is the core idea of **extrapolation**: it looks at how the approximations are improving and predicts what the result would be if we could continue this trend to a zero–error limit.

4 Romberg Integration (Richardson Extrapolation)

Romberg integration is a systematic way of applying extrapolation repeatedly to numerical integration. It starts from the composite trapezoidal rule and uses the predictable error behavior $E(h) \sim C\,h^2$ to eliminate lower–order terms step by step, achieving much higher accuracy with very little additional work.

4.1 The Core Idea: Cancelling Lower–Order Errors

The key insight of Romberg integration is very simple: if we know how the error behaves when we refine the step size, we can use that pattern to cancel it out.

Let us start with the trapezoidal rule. When we compute an integral with step h, the result $T(h)$ is close to the exact value I, but it still contains a small error that behaves like h^2:

$$T(h) = I + c\,h^2 + \text{smaller terms.}$$

If we make the step twice as fine (use $h/2$), we get a new result:

$$T(h/2) = I + c\,\frac{h^2}{4} + \text{smaller terms.}$$

Both approximations are almost correct, they just differ in how much of that $c\,h^2$ error they have. The first has all of it, the second only one quarter of it.

So what happens if we combine them? If we take $4T(h/2) - T(h)$, the $c\,h^2$ terms cancel perfectly:

$$4T(h/2) - T(h) = 3I + \text{tiny leftover terms.}$$

Dividing by 3 gives a much better estimate:

$$R_{2,1} = \frac{4T(h/2) - T(h)}{3}.$$

This new number has no h^2 error anymore, its error now starts with h^4. We have built a more accurate result *without* taking any new samples.

Note

Remember from last week: The trapezoidal rule is symmetric: each interior point appears twice in the sum, once as the right end of one subinterval and once as the left end of the next.

$$T(h) = \tfrac{h}{2}\big(f(a) + 2f(x_1) + 2f(x_2) + \cdots + f(b)\big)$$

Because the rule weights both sides equally, the local errors from adjacent trapezoids tend to cancel. As a result, the global error contains only even powers of h ($h^2, h^4, h^6, \dots$), the odd terms vanish due to this left–right balance.

The pattern continues. At the next level, we already have several estimates: each one still has some error left, but smaller and smaller. If we apply the same trick again, combining two results that differ by a factor of 4 in their error, we can cancel the next term too. That gives the general pattern:

$$\boxed{R_{\ell,k} = \frac{4^k R_{\ell,k-1} - R_{\ell-1,k-1}}{4^k - 1}.}$$

Each time we move one column to the right ($k \to k+1$), the order of accuracy goes up by two:

$$O(h^2) \;\to\; O(h^4) \;\to\; O(h^6) \;\to\; \dots$$

> **Note**
>
> The factor 4^k comes from the fact that the trapezoidal rule's error scales with h^2, and halving the step divides that error by 4. If we had a different method of order p, the same reasoning would use $(2^p)^k$ instead.

$k =$	1	2	3	4	5
Error term=	$\mathcal{O}(h^2)$	$\mathcal{O}(h^4)$	$\mathcal{O}(h^6)$	$\mathcal{O}(h^8)$	$\mathcal{O}(h^{10})$
j					
1	$I_{(1,1)}$ $h = h_1$	$I_{(1,2)} = \frac{4I_{(2,1)} - I_{(1,1)}}{3}$	$I_{(1,3)} = \frac{16I_{(2,2)} - I_{(1,2)}}{15}$	$I_{(1,4)} = \frac{64I_{(2,3)} - I_{(1,3)}}{63}$	$I_{(1,5)} = \frac{256I_{(2,4)} - I_{(1,4)}}{255}$
2	$I_{(2,1)}$ $h = \frac{h_1}{2}$	$I_{(2,2)} = \frac{4I_{(3,1)} - I_{(2,1)}}{3}$	$I_{(2,3)} = \frac{16I_{(3,2)} - I_{(2,2)}}{15}$	$I_{(2,4)} = \frac{64I_{(3,3)} - I_{(2,3)}}{63}$	
3	$I_{(3,1)}$ $h = \frac{h_1}{4}$	$I_{(3,2)}$	$I_{(3,3)}$		
4	$I_{(4,1)}$ $h = \frac{h_1}{8}$	$I_{(4,2)}$			
5	$I_{(5,1)}$ $h = \frac{h_1}{16}$				

Figure: Detailed Romberg integration table, each column cancels one more error term, increasing accuracy from $O(h^2)$ to $O(h^{10})$ (source: https://engcourses-uofa.ca/).

4.2 The Romberg Table: Structure and Interpretation

Romberg integration builds a triangular table of results. The first column contains trapezoidal values with finer and finer grids, and each subsequent column applies extrapolation to improve accuracy.

$$\begin{array}{llll} R_{1,0} & & & \\ R_{2,0} & R_{2,1} & & \\ R_{3,0} & R_{3,1} & R_{3,2} & \\ R_{4,0} & R_{4,1} & R_{4,2} & R_{4,3} \end{array}$$

The most accurate estimate at level ℓ is the value in the upper-right corner, $R_{\ell,\ell-1}$.

> **Note**
>
> The recursive relation for $R_{\ell,k}$ is the same Richardson extrapolation applied repeatedly, each time removing the next power of h^2 from the error term.

4.3 Numerical Example and Python Implementation

> **Note**
>
> In Python, Romberg integration can be implemented very efficiently: the same trapezoidal evaluations are reused at each refinement level, and extrapolation is computed using the recursive formula.

```python
import numpy as np

def f(x):
    return np.exp(-x**2)

def romberg(f, a, b, max_level=5):
    R = np.zeros((max_level, max_level))
    n = 1
    h = b - a
    R[0,0] = 0.5 * h * (f(a) + f(b))
    for i in range(1, max_level):
        n *= 2
        h /= 2
        x_mid = a + h * np.arange(1, n, 2)
        R[i,0] = 0.5 * R[i-1,0] + h * np.sum(f(x_mid))
        for k in range(1, i+1):
            R[i,k] = (4**k * R[i,k-1] - R[i-1,k-1]) / (4**k - 1)
    return R

R = romberg(f, 0, 1, 5)
print(R)
```

This simple implementation reproduces the classic Romberg table. Each refinement adds only a few new function evaluations, while the accuracy increases dramatically due to the extrapolation.

4.4 When Romberg Fails to Accelerate

Although Romberg integration is extremely efficient for smooth functions, it can fail to accelerate when:

- the function $f(x)$ is not sufficiently smooth (e.g., discontinuities or kinks),
- rounding errors dominate for very small h,

- or the integral has endpoint singularities.

In such cases, adaptive methods are usually preferred, since they focus computational effort where the function is irregular.

Note

Romberg integration shines when $f(x)$ is analytic and well-behaved, then the convergence is (almost) exponential. But for rough or non-smooth functions, it may offer little to no improvement over simpler composite rules.

5 Special Cases and Exponential Convergence

5.1 Why the Trapezoidal Rule Can Outperform Its Nominal Order

At first sight, the trapezoidal rule is a modest, second–order method: its error usually scales as $E \sim h^2$. But there is a surprising twist: under the right conditions, it can converge far faster than any power of h, sometimes approaching machine precision with only a few dozen points.

This happens in two "lucky" scenarios:

- When $f(x)$ is **periodic** on $[a, b]$, so that $f(a) = f(b)$ and all derivatives match at the ends. The boundary errors that normally dominate simply cancel.
- When $f(x)$ is **analytic** (infinitely smooth and extendable to the complex plane), so that the remaining error terms decay geometrically rather than algebraically.

The intuition. To understand why the trapezoidal rule sometimes performs far better than expected, let us recall what its error actually comes from.

Each trapezoid approximates a small portion of the curve by a straight line. If the function is gently curved, this straight line misses the true area by a tiny amount, a small "wedge" of error near the edges of the interval. When we sum many trapezoids, these wedges normally add up: each one leaves a small leftover area, and all those pieces accumulate, giving the familiar $O(h^2)$ global error.

Now imagine a different situation. Suppose the function lines up perfectly at the endpoints, so that $f(a) = f(b)$ and even the slopes are the same. This happens when the function is **periodic**. In this case, the trapezoid at the beginning and the trapezoid at the end have edge errors that are mirror images of each other. When we add everything together, those endpoint errors cancel out almost exactly, so there is no mismatch at the edges to accumulate anymore.

With the boundary effects gone, what remains is the smooth, continuous curvature of the function inside the interval. If the function is also **analytic** (infinitely smooth), this curvature behaves in a very regular and predictable way: it can be represented almost perfectly by a sum of sine and cosine waves. Each time we refine the grid, these waves are captured more and more accurately, and the error shrinks at an almost geometric rate. That is why, instead of decreasing like h^2, it can fall like q^N for some $0 < q < 1$, an exponential decay.

In simpler words: the trapezoidal rule becomes "super-accurate" when the function is so smooth and well-behaved that it offers no surprises to the straight lines connecting its points. Refining the grid no longer just improves the resolution, it wipes out whole layers of error at once.

> Note
>
> In these ideal cases, halving h does not merely divide the error by 4, it can reduce it by factors of 10^3, 10^6, or more. This remarkable behavior is called **spectral** or **exponential convergence**.

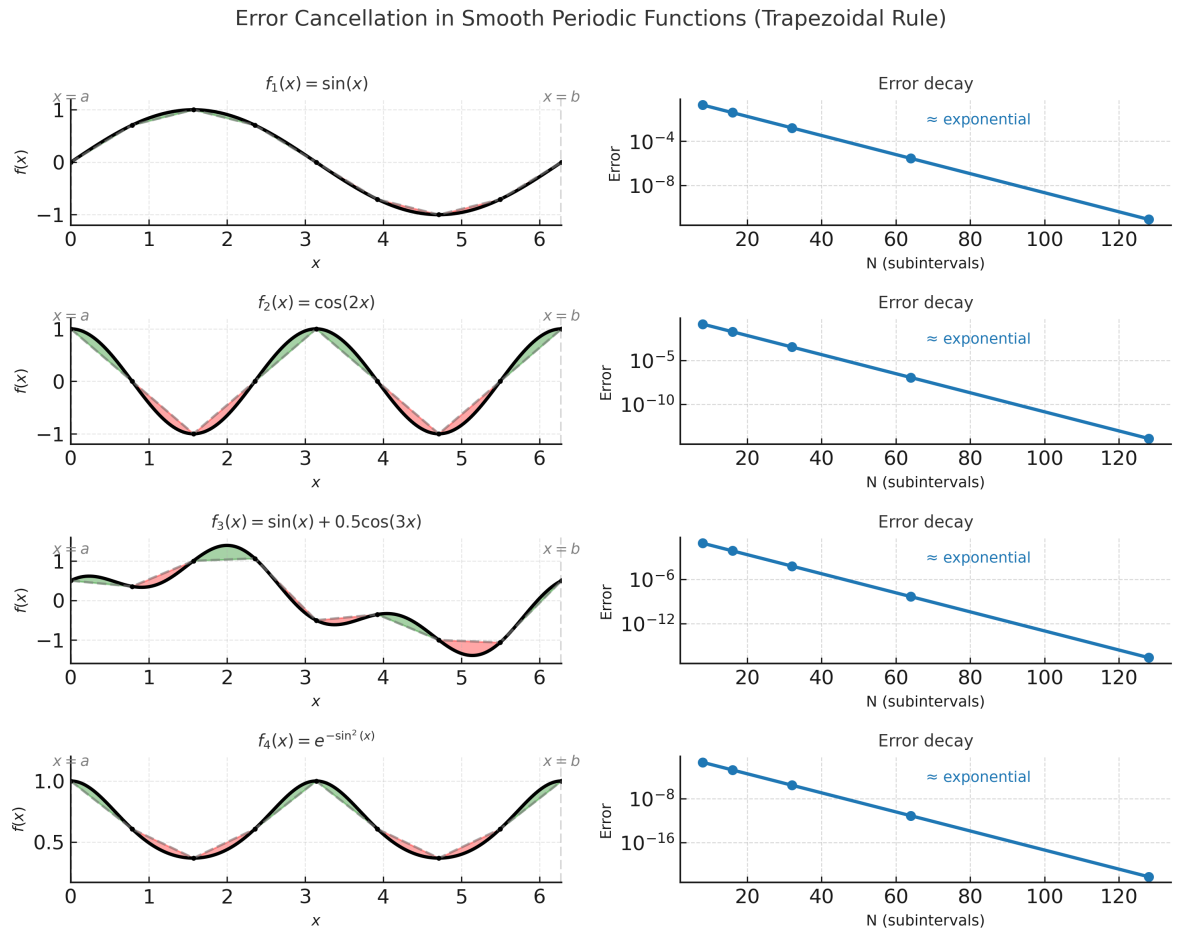

Figure: Local trapezoidal errors (red and green) cancel across subintervals for periodic functions, producing nearly perfect error elimination and exponential convergence.

5.2 Numerical Experiment: Periodic vs. Non–Periodic

To see this effect in practice, compare two smooth integrands on the same interval:

$$\int_0^{2\pi} \sin(x)\,dx = 0, \qquad \int_0^{2\pi} \left(e^x + \sin x\right) dx = e^{2\pi} - 1.$$

Both are smooth, but only the first is perfectly periodic.

N	Periodic $\sin(x)$	Non–periodic $e^x + \sin(x)$
10	10^{-7}	10^{-1}
20	10^{-15}	10^{-3}
40	10^{-16}	10^{-5}

Note

For the periodic case, the function and all its derivatives match at $x = 0$ and $x = 2\pi$, so the edge errors cancel almost perfectly — producing exponential convergence. In contrast, for the non–periodic case $f(x) = e^x + \sin(x)$, the endpoints differ by several orders of magnitude. That mismatch prevents cancellation and leaves the trapezoidal rule at its standard $O(h^2)$ accuracy.

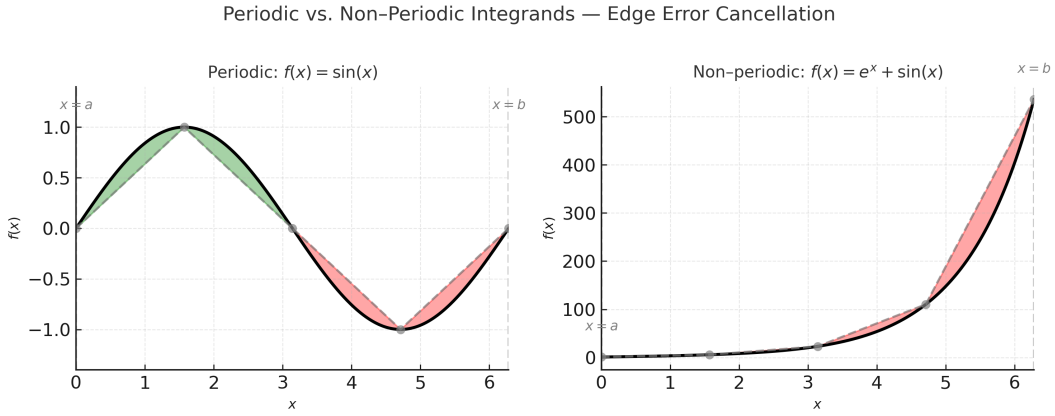

Figure: Periodic vs. non–periodic integrands, for $f(x) = \sin(x)$ the edge errors cancel, while for $f(x) = e^x + \sin(x)$ they accumulate, leading to much slower convergence.

6 Spectral and Clenshaw–Curtis Quadrature

6.1 From Polynomial to Trigonometric Approximations

Traditional quadrature rules approximate the integrand $f(x)$ by a polynomial on each interval. Spectral methods take a more global view: they approximate $f(x)$ by a smooth trigonometric expansion that captures the overall shape of the function across the entire domain.

This shift from piecewise to global approximation is what allows spectral methods to achieve almost miraculous accuracy for analytic functions. The integration error decreases faster than any power of h.

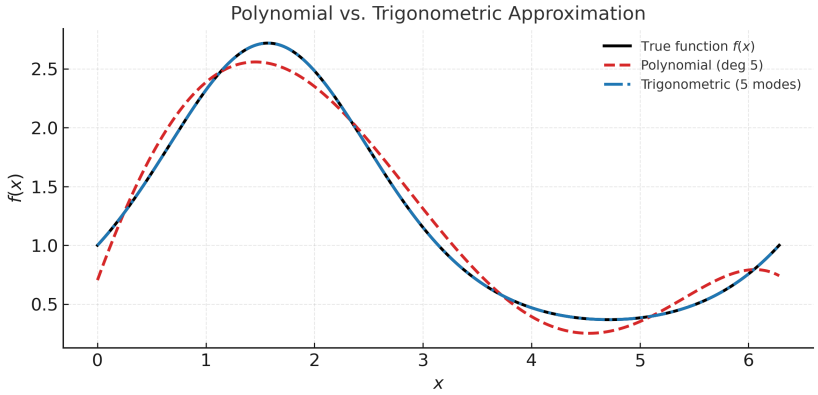

Figure: Polynomial vs. trigonometric basis: global, smooth approximations enable much faster error decay for analytic functions.

6.2 Fast Computation Using FFT for Periodic Functions

At the heart of spectral quadrature lies a simple observation: if a function $f(x)$ is smooth (or periodic), it can be represented as a sum of sine and cosine waves. Each wave has a frequency and amplitude, and integrating $f(x)$ is equivalent to summing up the contributions of all those waves.

$$f(x) \approx \sum_{k=-N/2}^{N/2} \hat{f}_k \, e^{ikx}, \qquad I = \int_a^b f(x)\,dx \approx \sum_{k=-N/2}^{N/2} \hat{f}_k \int_a^b e^{ikx}\,dx.$$

Only the term with $k = 0$ survives the integration, because all other oscillatory terms average out to zero:

$$\int_a^b e^{ikx}\,dx = \begin{cases} b-a, & \text{if } k = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Hence,

$$I \approx (b-a)\,\hat{f}_0,$$

where $\hat{f}_0$ is simply the mean (zero–frequency component) of $f(x)$.

Connecting to the FFT. The coefficients $\hat{f}_k$ are precisely what the **Discrete Fourier Transform (DFT)** computes. Instead of evaluating these coefficients directly (which would take $O(N^2)$ operations), we use the **Fast Fourier Transform (FFT)**, which reduces the cost

to $O(N \log N)$.

Once the FFT has computed all frequency components of $f(x)$, extracting the integral becomes trivial: take the first coefficient $\hat{f}_0$ and scale it by the interval length. This means the entire integration process, from sampling to result, can be performed almost instantaneously even for thousands of grid points.

Note

The FFT acts like a "smart summation" engine: it reorganizes the N pointwise samples $f(x_k)$ into a set of coefficients $\hat{f}_k$ that represent the strength of each sine and cosine wave present in the signal.
Among these coefficients, $\hat{f}_0$ corresponds to the *average value* (the zero–frequency component) of the function. When we integrate a periodic function over one full cycle, all oscillatory parts cancel out, and only this mean value remains. That is why numerical integration using the Fourier representation essentially reduces to extracting $\hat{f}_0$ and scaling it by the interval length $(b-a)$:

$$I \approx (b-a)\,\hat{f}_0.$$

In discrete form, this is equivalent to forming a particular weighted sum of the sampled values:

$$I \approx \sum_{k=0}^{N-1} w_k\, f(x_k).$$

The quadrature weights w_k describe how each sample contributes to the zero–frequency component. Rather than computing them analytically, they can be obtained numerically and very efficiently through an **inverse FFT** applied to the first Fourier mode.
In other words, FFT–based quadrature computes the integral by converting $f(x)$ into frequency space, then using the frequency–zero component to reconstruct the correct weighted sum in the spatial domain. It is both mathematically elegant and computationally efficient.

Practical view. In modern scientific computing, FFT-based quadrature appears everywhere:

- in **computational physics**, for fast evaluation of convolutions and spectra;
- in **signal processing**, where integration corresponds to energy or power computation;
- and in **PDE solvers**, where smooth periodic solutions make FFT integration ideal.

This tight link between smoothness, periodicity, and computational speed is what makes spectral methods uniquely powerful.

6.3 Optional — Clenshaw–Curtis Quadrature

A closely related and remarkably elegant approach is **Clenshaw–Curtis quadrature**. Instead of sampling $f(x)$ at equally spaced points, it evaluates the function at *Chebyshev nodes*:

$$x_j = \cos\Big(\frac{j\pi}{N}\Big), \qquad j = 0, 1, \ldots, N.$$

These points are distributed non–uniformly: they cluster near the interval endpoints $x = \pm 1$ and are more sparse in the center. At first, this may seem odd, but it turns out to have major advantages.

1. Why clustering helps. Near the edges of the interval, polynomial interpolation tends to oscillate strongly (the Runge phenomenon). By placing more points where the function is harder to approximate, Chebyshev nodes control this instability and produce much smoother interpolations and more accurate integrals.

2. Connection to Fourier methods. The Chebyshev nodes are essentially the cosine of uniformly spaced angles. This trigonometric structure allows the entire integration process to be reformulated as a **cosine transform**. In practice, this means that the integration weights can be computed using an **FFT**, just like in the spectral quadrature discussed earlier. Hence, Clenshaw–Curtis can achieve spectral–like accuracy while remaining extremely fast.

3. Accuracy and stability. For analytic functions, the error decreases almost exponentially with the number of nodes N, a hallmark of **spectral accuracy**. In numerical tests, Clenshaw–Curtis often matches or even exceeds the performance of any polynomial-version quadrature, especially when implemented in finite–precision arithmetic, because it avoids ill–conditioned polynomial systems.

Practical intuition. In summary:

- The nodes x_j are the projections of equally spaced angles on the unit circle.
- The integration weights w_j can be computed using a discrete cosine transform (DCT), which is implemented via FFT.
- The result is a quadrature rule that converges nearly exponentially for smooth $f(x)$, yet is easy to implement and very stable.

For these reasons, Clenshaw–Curtis quadrature is often the default choice in modern spectral and pseudospectral solvers.

6.4 When Spectral Methods Are Worth Using

Spectral or FFT–based quadrature is particularly advantageous when:

- $f(x)$ is periodic or analytic (infinitely differentiable);
- high precision is required with only a few evaluations;

- or the same integrand is evaluated many times, for instance, inside time-stepping PDE solvers or control simulations.

For less smooth functions or integrals with singularities, adaptive composite methods (like Romberg) remain the more robust and reliable choice.